

Report

Team Members

- Jayesh Pandit IMT2023111
- Lakshya Kapoor IMT2023509
- Sahas Sangal IMT2023556

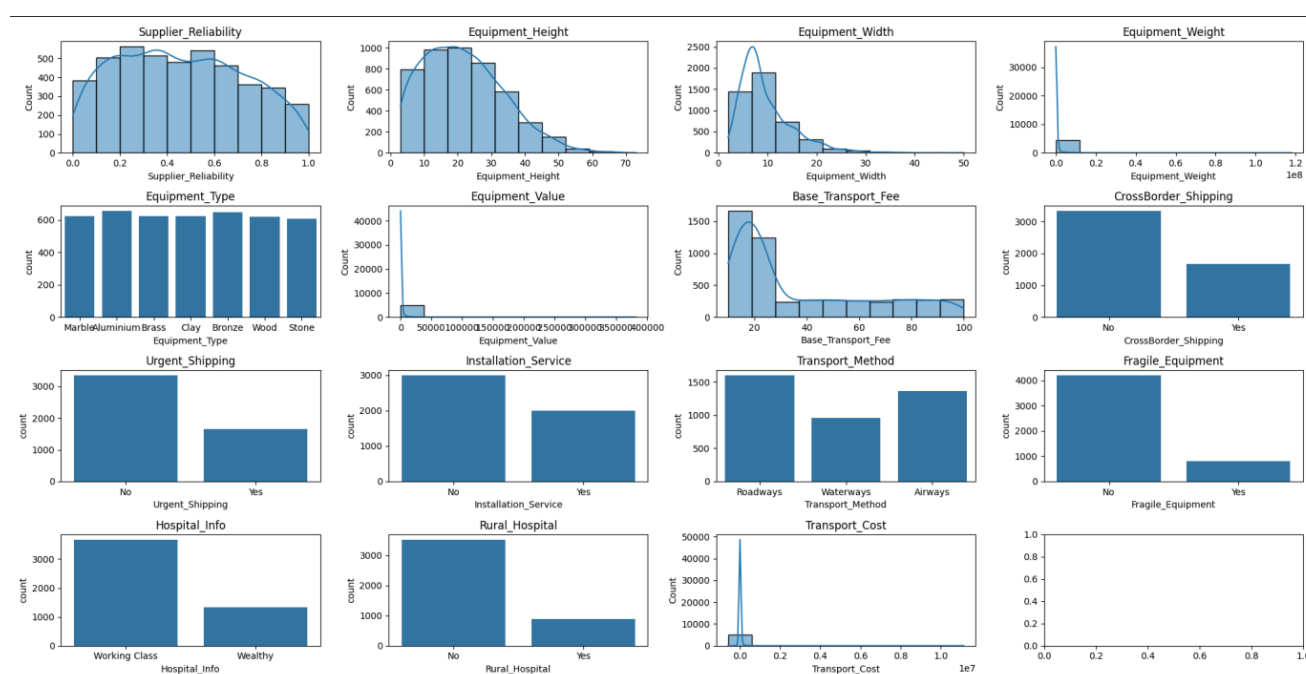
GITHUB_LINK : https://github.com/Sahas-out/Medical_Equipment_TransportCost_Prediction

Task

The goal of this project is to develop a machine learning model that accurately predicts the **transport cost** of medical equipment deliveries based on various logistical, supplier, and equipment-related factors. The dataset contains detailed information about hospitals, suppliers, equipment specifications, transport methods, and delivery conditions.

EDA

Histogram plots for various columns

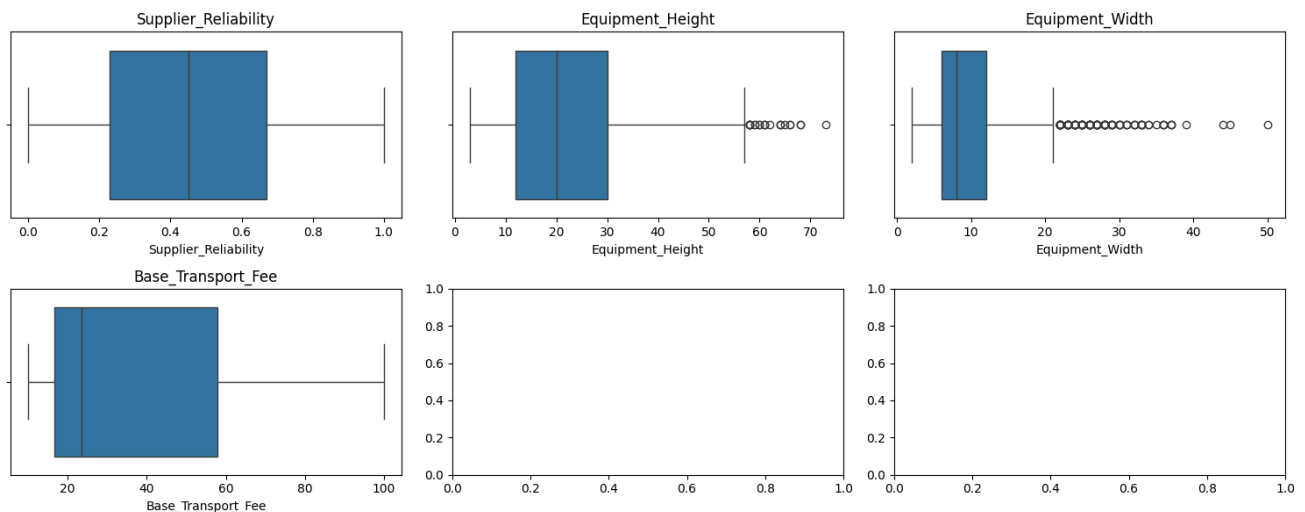


Distribution Insights:

The variables Supplier_Reliability, Equipment_Height, and Equipment_Width exhibit approximately normal distributions. The categorical variables show balanced class distributions, with no dominant category. In contrast, Equipment_Value, Equipment_Weight,

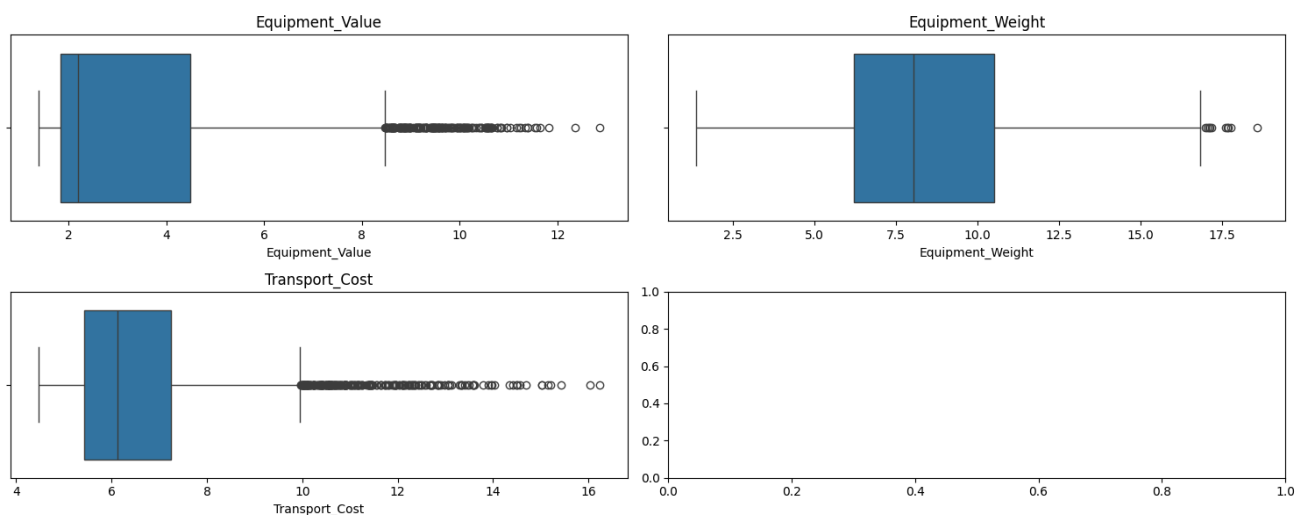
and Transport_Cost display skewed distributions, where a large proportion of values fall within a narrow range and the remaining values are widely dispersed

boxplots for numerical Columns



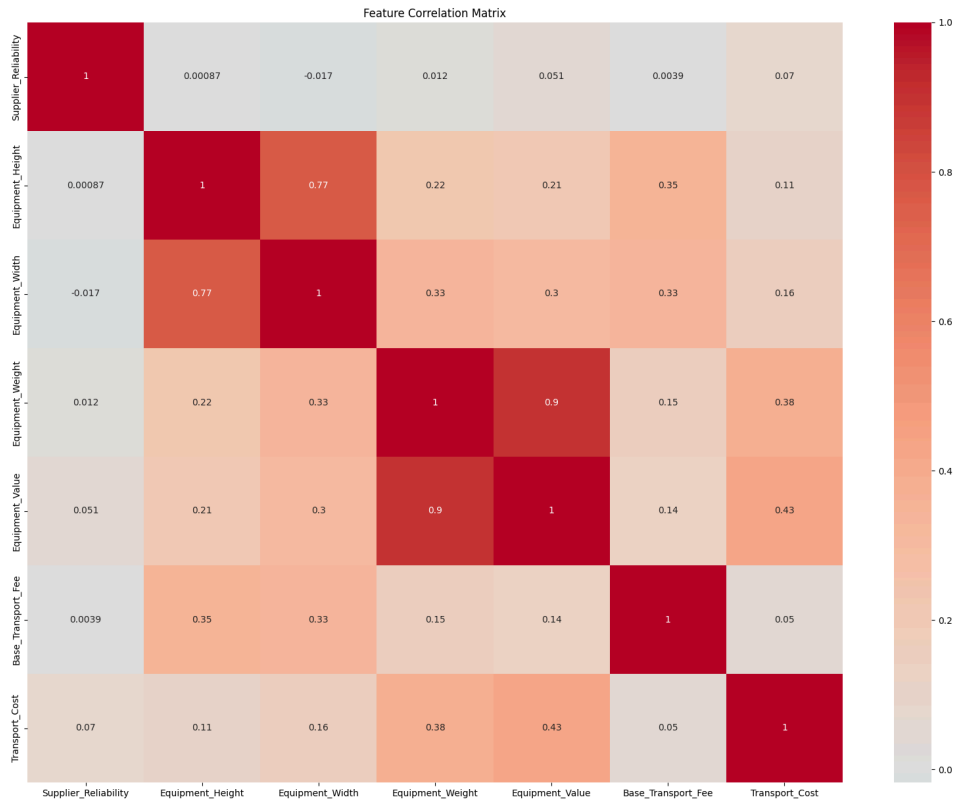
- From the boxplots above we can see there are not many outliers for Supplier_Reliability, Equipment_Height, Equipment_Width, Base_Transport_Fee

Boxplot for numerical columns with log transform(for better readability)



- from the above boxplots one can see Equipment_Value, Equipment_Weight and Transport_Cost have high number of outliers

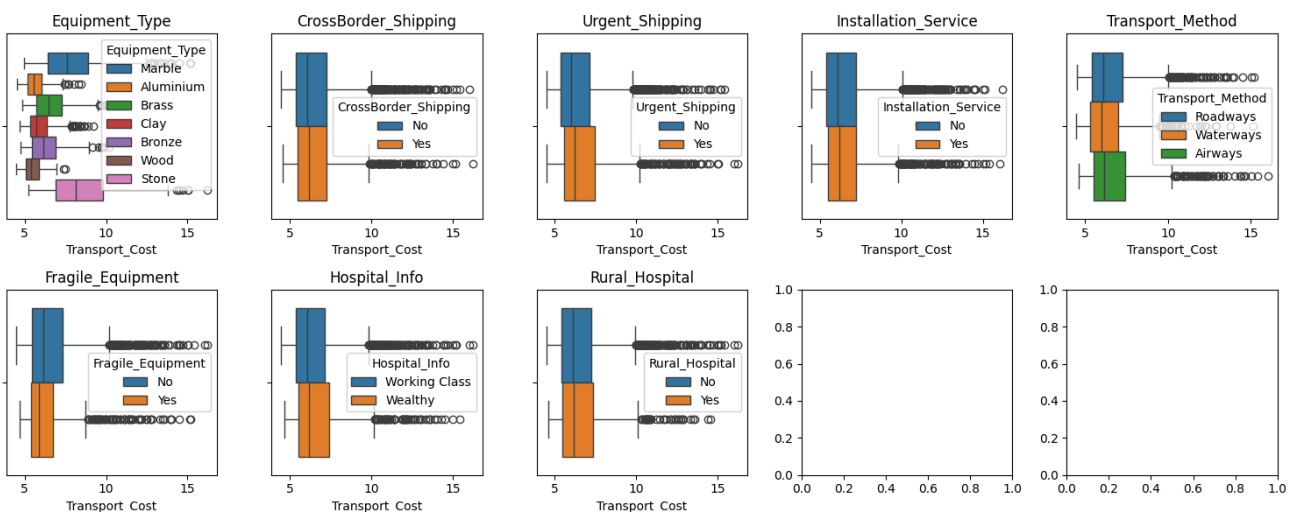
Correlation Heatmap between numerical Columns



Correlation Analysis:

A strong positive correlation is observed between Equipment_Width and Equipment_Height, as well as between Equipment_Value and Equipment_Weight. The target variable Transport_Cost shows a moderate correlation with both Equipment_Value and Equipment_Weight, indicating that heavier and more valuable equipment tends to incur higher transport costs.

Boxplots of Transport Cost with hue as different categorical columns



Feature Importance:

The features CrossBorder_Shipping, Urgent_Shipping, Installation_Service,

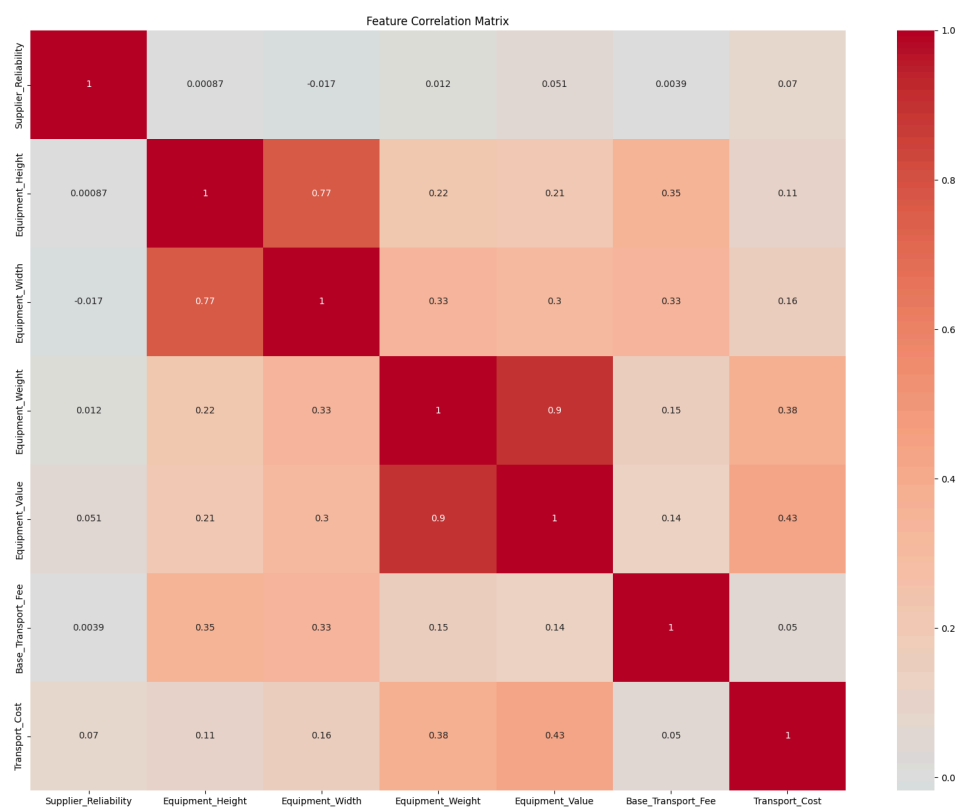
Transport_Method, Fragile_Equipment, Hospital_Info, and Rural_Hospital contribute relatively little to explaining the variance in Transport_Cost. In contrast, Equipment_Type shows a moderate influence on the target variable.

Preprocessing

Null-Percentage Values for columns with missing values

Feature Name	Null_percentage (%)
Transport_Method	21.42
Equipment_Type	11.98
Supplier_Reliability	11.74
Rural_Hospital	11.72
Equipment_Weight	9.20
Equipment_Width	8.86
Equipment_Height	5.66

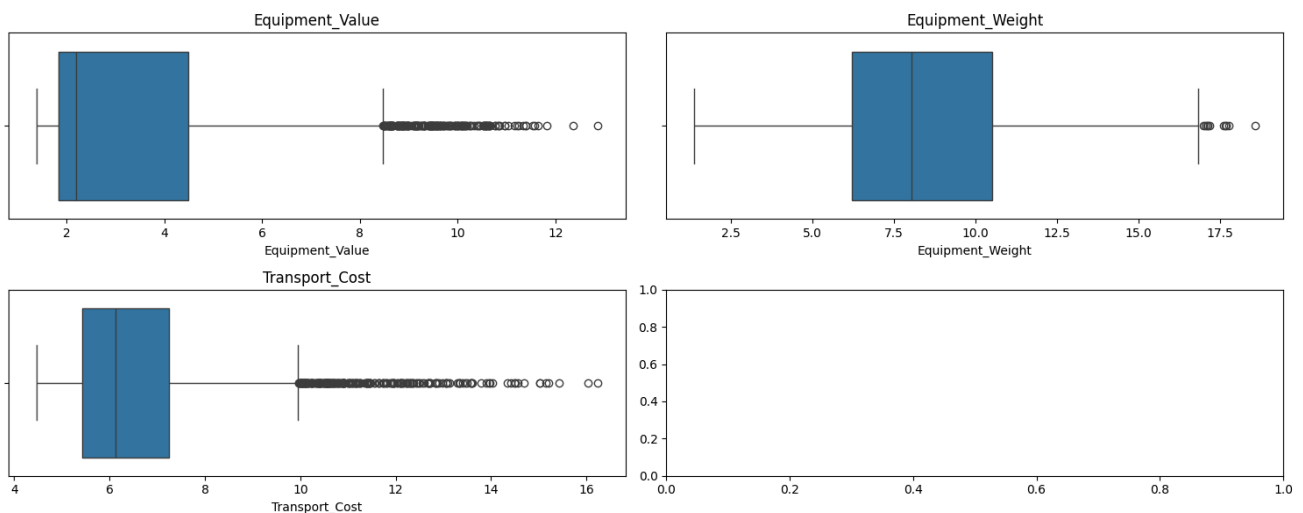
correlation heatmap



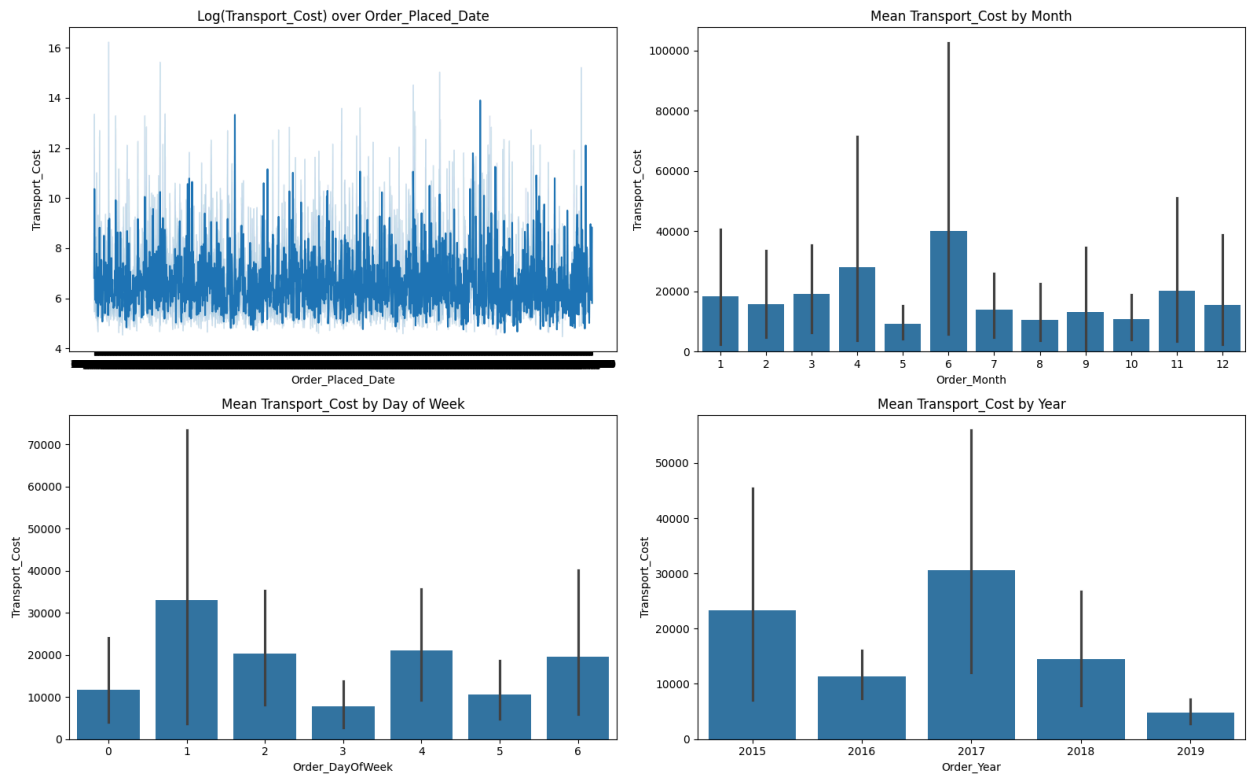
- **Imputing Numerical Columns:** Due to strong correlations between certain numerical features (Equipment_Width–Equipment_Height and Equipment_Weight–Equipment_Value), an Iterative Imputer was used to handle missing values. This method leverages regression models to estimate missing entries more accurately based on relationships among numerical variables.
- **Imputing Categorical Columns:** we imputed the null values of categorical columns with "Unknown" we opted for simple imputation.
- **Encoding Categorical Columns** One-Hot Encoding was applied to categorical features, as it effectively represents non-ordinal categories without introducing any artificial ordering. This method was well-suited for the dataset since none of the categorical features contained an excessively large number of unique classes.
- **Dropping Columns** We dropped columns
 - supplier_name
 - hospital_id,
 - supplier_location,
 - order_placed_date
 - delivery_date
 - hospital_location

These columns have over 4900 unique values making them useless for training

Feature Engineering



- **Transport_Cost transform :** as seen through the box plot there were large number of outliers in the transport_cost column so we decided to train our model on log of transport_values for reducing the sensitivity of model to outliers
- **new features using Order_Placed_Date and Delivery_Date :** we created the following new features that we thought would be helpful for model training cause of the above figure



1. Order Year

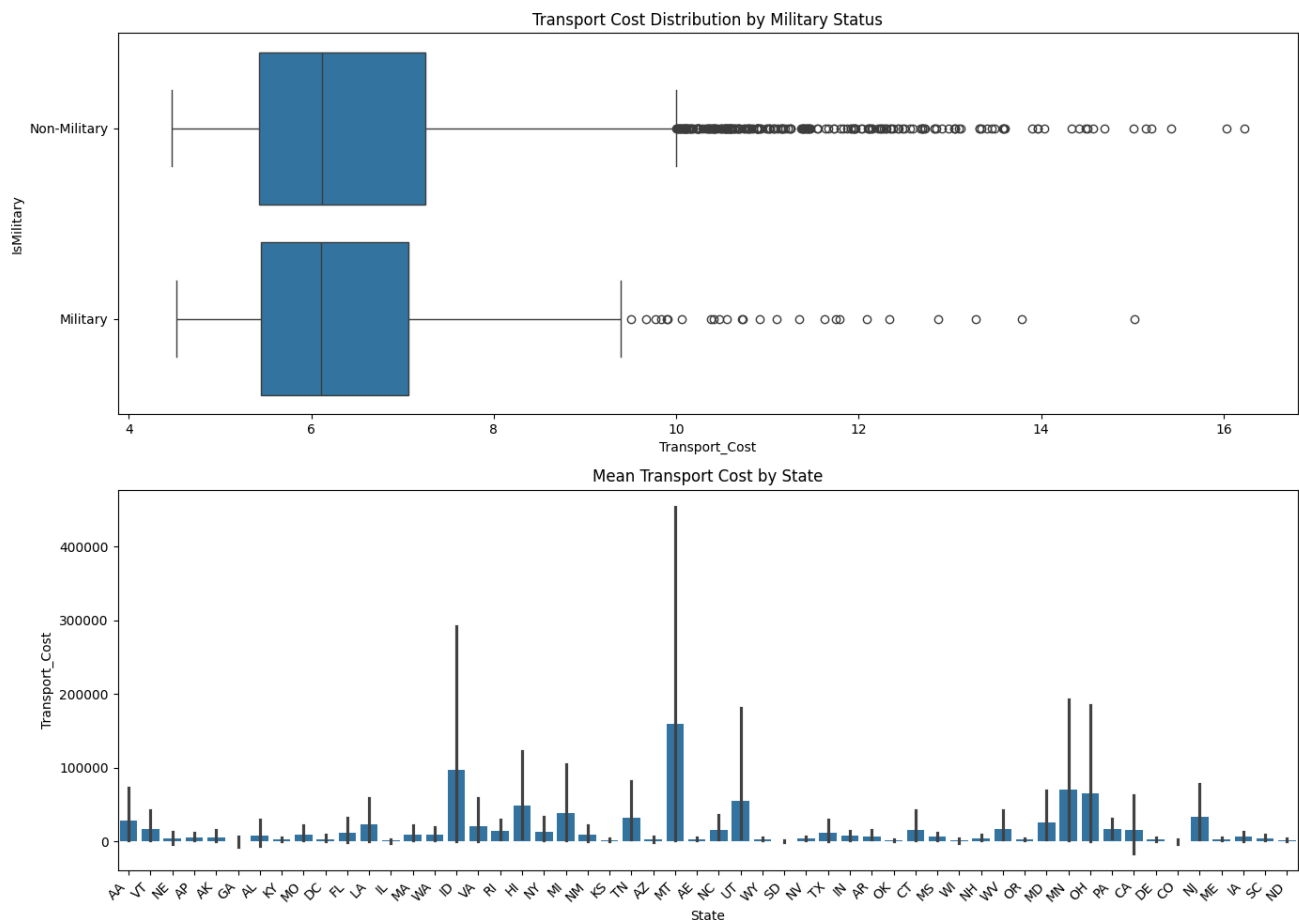
2. Order Month

3. Order DayOfWeek

4. Delivery_Time

- *new features using Hospital_Location*

as can be concluded from above graph we thought the following would be helpful in model training



as can be concluded from above graph we thought the following would be helpful in model training

- Hospital State
- Is Military

- *new features derived from equipment features*

the following we thought would be useful for training of model that were derived from equipment features

- Equipment_Area
- Equipment_Density
- Equipment_Cost_Per_Area
- Equipment_Cost_per_Weight

Model Training

Model	Kaggle Score	CV RMSE	Validation MSE	Notes / Best Params
xgboost	6593135201.986	0.2313	0.0106	learning_rate=0.01, max_depth=3, n_estimators=100
simple linear	4342449898.947	—	0.0031	—
ridge	5393062469.072	0.1914	0.0026	alpha=1000.0

Model	Kaggle Score	CV RMSE	Validation MSE	Notes / Best Params
random forest	5176375856.251	0.2263	0.0028	max_depth=10, max_features=sqrt, min_samples_split=5
polynomial	4410934655.553	0.2320	0.0031	poly degree=1, features expanded to 42
lasso	4138549265.046	0.1860	0.0027	alpha=10.0, features selected: 3
knn	5081359181.497	0.2220	0.0026	metric=manhattan, n_neighbors=15, weights=uniform
elasticnet	4138554001.507	0.1860	0.0027	alpha=100.0, l1_ratio=0.1, features selected: 3
decision tree	6688577760.162	0.2214	0.0048	max_depth=10, min_samples_leaf=10
adaboost	8632453243.297	0.1822	0.0030	learning_rate=0.1, n_estimators=50, loss=square

We used GridSearch cv in all the models to hypertune the parameters and used the same preprocessing as mentioned in preprocessing section

Discussion on the Performance of Different Approaches

Our goal was to develop regression models that minimized prediction error (RMSE/MSE) and achieved lower Kaggle scores. Several algorithms were tested, each showing different strengths and trade-offs.

1. Lasso and ElasticNet:

These models achieved the **best overall performance**, with low validation MSE (~ 0.0027) and the **lowest Kaggle scores** ($\sim 4.1e9$). Their strength came from effective regularization and feature selection—both models retained only three key features, enhancing generalization and reducing noise.

2. Ridge Regression:

Ridge also performed strongly (MSE 0.0026) by handling multicollinearity well with `alpha=1000`. It provided a good balance between bias and variance, though slightly behind Lasso and ElasticNet.

3. KNN and Random Forest:

Both models delivered competitive results (MSE $\approx 0.0026-0.0028$), showing their capability to capture nonlinear patterns. However, they were more sensitive to parameter tuning (e.g., neighbors in KNN or depth in Random Forest) and slightly less consistent across validation and Kaggle scores.

4. Linear and Polynomial Regression:

These served as baseline models. While they produced reasonable MSEs (~ 0.0031), their

simplicity limited their ability to model complex relationships. Polynomial expansion did not yield significant gains.

5. XGBoost, Decision Tree, and AdaBoost:

These ensemble methods showed **higher Kaggle errors (6.6e9–8.6e9)**, suggesting potential **overfitting** despite parameter tuning. Although XGBoost captured complex interactions, its generalization to unseen data was weaker compared to simpler, regularized linear models.

Interpretation of Top Performers (Lasso & ElasticNet)

- **Regularization Power:** The L1/L2 penalties effectively reduced overfitting and improved generalization.
- **Feature Selection:** Limiting to only the most predictive features enhanced model simplicity and performance.
- **Consistency:** These models achieved both the lowest MSEs and the best Kaggle results, indicating robustness across data splits.

Interesting Observations

- **Simplicity Outperformed Complexity:** Linear regularized models surpassed advanced ensembles, proving that fewer, cleaner features can outperform deep models on smaller or noisier datasets.
- **Overfitting in Boosted Models:** High Kaggle errors from XGBoost and AdaBoost reflected their sensitivity to noise and tendency to overfit despite strong validation scores.
- **Importance of Tuning:** Models with careful GridSearchCV tuning—especially Ridge and KNN—showed measurable improvements in both validation and Kaggle results.
- **Feature Engineering Matters:** Removing uninformative or redundant features substantially improved model stability and accuracy.